Object-Oriented Programming in Python

A Comprehensive Guide

Slide 1: Introduction to OOP

What is Object-Oriented Programming?

- Programming paradigm based on "objects"
- Objects contain data (attributes) and code (methods)
- Focuses on organizing code into reusable components
- Python fully supports OOP concepts

Slide 2: Why Use OOP?

Benefits:

- **Modularity** - Code is organized into separate classes
- **Reusability** - Classes can be reused across projects
- **Maintainability** - Easier to update and debug
- **Abstraction** - Hide complex implementation details
- **Encapsulation** - Bundle data and methods together

Slide 3: Classes in Python

What is a Class?

- Blueprint or template for creating objects
- Defines attributes (data) and methods (functions)
- Think of it as a cookie cutter, objects are the cookies

Syntax:

```
class ClassName:
    # class body
    pass
```

Slide 4: Defining a Simple Class

```
class Dog:
    pass
```

Creating an object (instance):

```
my_dog = Dog()
```

- `Dog` is the class
- `my_dog` is an instance/object of the Dog class

Slide 5: The `__init__` Method

Constructor Method:

- Special method called when object is created
- Initializes object attributes
- Always takes `self` as first parameter

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

Slide 6: Understanding `self`

What is `self`?

- Reference to the current instance
- Must be first parameter in instance methods
- Used to access attributes and methods of the class

```
class Dog:
    def __init__(self, name):
        self.name = name  # self refers to this instance
```

Slide 7: Defining Attributes

Two types of attributes:

1. **Instance Attributes** - Unique to each object

```
class Dog:
    def __init__(self, name):
        self.name = name  # Instance attribute
```

2. **Class Attributes** - Shared by all objects

```
class Dog:
    species = "Canis familiaris"  # Class attribute
```

Slide 8: Instance vs Class Attributes

```
class Dog:
    species = "Canis familiaris"  # Class attribute

    def __init__(self, name, age):
        self.name = name      # Instance attribute
        self.age = age        # Instance attribute

dog1 = Dog("Buddy", 3)
dog2 = Dog("Max", 5)

print(dog1.species)  # "Canis familiaris" (shared)
print(dog1.name)     # "Buddy" (unique)
```

Slide 9: Defining Methods

Instance Methods:

- Functions defined inside a class
- Always take `self` as first parameter
- Can access and modify instance attributes

```
class Dog:
    def __init__(self, name):
        self.name = name

    def bark(self):
        return f"{self.name} says Woof!"
```

Slide 10: Calling Methods

```
class Dog:
    def __init__(self, name):
        self.name = name

    def bark(self):
```

```
        return f"{self.name} says Woof!"

# Create object
my_dog = Dog("Buddy")

# Call method
print(my_dog.bark())  # Output: Buddy says Woof!
```

Slide 11: Methods with Parameters

```
class Calculator:
    def add(self, a, b):
        return a + b

    def multiply(self, a, b):
        return a * b

calc = Calculator()
print(calc.add(5, 3))       # Output: 8
print(calc.multiply(4, 7))  # Output: 28
```

Slide 12: Class Methods

Using `@classmethod` decorator:

- Takes `cls` as first parameter (not `self`)
- Can access/modify class state
- Can be called on class itself

```
class Dog:
    count = 0

    @classmethod
    def increment_count(cls):
        cls.count += 1

Dog.increment_count()
print(Dog.count)  # Output: 1
```

Slide 13: Static Methods

Using `@staticmethod` decorator:

- Doesn't take `self` or `cls` parameter

- Can't access instance or class attributes
- Utility functions related to the class

```
class Math:
    @staticmethod
    def is_even(num):
        return num % 2 == 0

print(Math.is_even(4))  # Output: True
```

Slide 14: Public Attributes

Default in Python:

- All attributes are public by default
- Can be accessed from anywhere

```
class Person:
    def __init__(self, name):
        self.name = name  # Public attribute

person = Person("Alice")
print(person.name)  # Accessible
person.name = "Bob" # Can be modified
```

Slide 15: Protected Attributes

Convention: Single underscore _

- Indicates "internal use"
- Not enforced by Python
- Convention suggests: "don't access directly"

```
class Person:
    def __init__(self, name):
        self._name = name  # Protected attribute

person = Person("Alice")
print(person._name)  # Still accessible, but discouraged
```

Slide 16: Private Attributes

Convention: Double underscore __

- Name mangling applied
- Harder to access from outside
- Python renames to `_ClassName__attribute`

```
class Person:
    def __init__(self, name):
        self.__name = name  # Private attribute

person = Person("Alice")
# print(person.__name)  # AttributeError
print(person._Person__name)  # Still accessible via mangling
```

Slide 17: Encapsulation with Getters/Setters

```
class Person:
    def __init__(self, name):
        self.__name = name

    def get_name(self):
        return self.__name

    def set_name(self, name):
        if len(name) > 0:
            self.__name = name

person = Person("Alice")
print(person.get_name())  # Alice
person.set_name("Bob")
print(person.get_name())  # Bob
```

Slide 18: Property Decorators

Pythonic way: `@property`

```
class Person:
    def __init__(self, name):
        self._name = name

    @property
    def name(self):
        return self._name

    @name.setter
    def name(self, value):
        if len(value) > 0:
```

```
            self._name = value

person = Person("Alice")
print(person.name)      # Uses getter
person.name = "Bob"     # Uses setter
```

Slide 19: Scope Summary Table

Notation	Type	Access Level	Example
`name`	Public	Everywhere	`self.name`
`_name`	Protected	Internal (convention)	`self._name`
`__name`	Private	Name mangled	`self.__name`

Note: Python doesn't enforce true privacy - it's based on convention and trust.

Slide 20: Complete Example - Bank Account

```
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner           # Public
        self._account_id = "12345"   # Protected
        self.__balance = balance     # Private

    @property
    def balance(self):
        return self.__balance

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
            return True
        return False

    def withdraw(self, amount):
        if 0 < amount <= self.__balance:
            self.__balance -= amount
            return True
        return False
```

Slide 21: Using the BankAccount Class

```
# Create account
account = BankAccount("Alice", 1000)
```

```
# Access public attribute
print(account.owner)  # Alice

# Use methods
account.deposit(500)
account.withdraw(200)

# Access balance via property
print(account.balance)  # 1300

# Cannot directly modify private attribute
# account.__balance = 0  # Won't work
```

Slide 22: Method Types Comparison

```
class Example:
    class_var = "shared"

    def __init__(self, value):
        self.value = value

    def instance_method(self):
        # Can access instance and class attributes
        return f"{self.value} and {self.class_var}"

    @classmethod
    def class_method(cls):
        # Can access class attributes only
        return cls.class_var

    @staticmethod
    def static_method(x):
        # No access to instance or class
        return x * 2
```

Slide 23: Best Practices

1. **Use meaningful class names** (PascalCase)
2. **Use `self` consistently** for instance reference
3. **Initialize attributes in `__init__`**
4. **Follow naming conventions:**
   - Public: `attribute`
   - Protected: `_attribute`
   - Private: `__attribute`
5. **Use properties instead of getters/setters**
6. **Keep classes focused** (Single Responsibility)

Slide 24: Common Mistakes to Avoid

❌ **Forgetting `self`:**

```
class Dog:
    def bark():  # Missing self
        return "Woof"
```

❌ **Directly accessing private attributes:**

```
person.__name = "Bob"  # Creates new attribute, doesn't modify
```

✓ **Correct approach:**

```
person.set_name("Bob")  # Use method or property
```

Slide 25: Practical Example - Student Class

```
class Student:
    school = "ABC High School"  # Class attribute

    def __init__(self, name, grade):
        self.name = name           # Public
        self._grade = grade        # Protected
        self.__id = None           # Private

    def study(self, subject):
        return f"{self.name} is studying {subject}"

    @property
    def grade(self):
        return self._grade

    @grade.setter
    def grade(self, value):
        if 0 <= value <= 100:
            self._grade = value

    @classmethod
    def change_school(cls, new_school):
        cls.school = new_school
```

Slide 26: Using the Student Class

```
# Create students
student1 = Student("Alice", 95)
student2 = Student("Bob", 87)

# Call instance method
print(student1.study("Math"))  # Alice is studying Math

# Access property
print(student1.grade)  # 95
student1.grade = 98    # Use setter
print(student1.grade)  # 98

# Access class attribute
print(Student.school)  # ABC High School

# Call class method
Student.change_school("XYZ High School")
print(student1.school)  # XYZ High School
```

Slide 27: Summary - Key Concepts

Classes:

- Defined with `class` keyword
- Blueprint for objects

Attributes:

- Instance attributes (unique per object)
- Class attributes (shared across objects)

Methods:

- Instance methods (work with object data)
- Class methods (work with class data)
- Static methods (utility functions)

Slide 28: Summary - Scope & Access

Public: No underscore

- Accessible everywhere
- Default behavior

Protected: Single underscore _

- Convention for internal use

- Not enforced

Private: Double underscore `__`

- Name mangling applied
- Harder to access externally

Slide 29: Further Learning

Advanced OOP Topics:

- Inheritance
- Polymorphism
- Abstract classes
- Multiple inheritance
- Magic methods (`__str__`, `__repr__`, etc.)
- Descriptors
- Metaclasses

Slide 30: Conclusion

Object-Oriented Programming in Python:

- ✓ Easy to learn syntax
- ✓ Flexible and powerful
- ✓ Supports all OOP principles
- ✓ Convention-based privacy
- ✓ Pythonic approach with properties

Remember: Python's philosophy is "We're all consenting adults" - privacy is based on trust and convention, not enforcement.

Thank You!

Questions?